

CI/CD & Test-Harness: Hướng dẫn chi tiết

<https://gemini.google.com/share/359be4d7b142>



Được tạo bằng 3.1 Pro lúc 22:30 3 tháng 7, 2026 • Đã xuất bản vào lúc 00:42 4 tháng 7, 2026

Chào bạn, bạn hãy cho tôi biết, CI/CD & Test-Harness Engineering là gì? Vì đây là chủ đề nhóm tôi cần tìm hiểu cho môn: "Kiểm thử phần mềm", nên sau khi bạn cho tôi biết chủ đề trên là gì, bạn hãy đề xuất các nội dung trong b

Chào bạn, đây là một chủ đề cực kỳ thú vị, rất sát với thực tế ngành công nghiệp phần mềm và là bước đệm tuyệt vời nếu bạn đang muốn tìm hiểu sâu hơn về tư duy hệ thống cũng như văn hóa DevOps.

Dưới đây là lời giải thích đơn giản, rõ ràng về chủ đề, cùng với cấu trúc báo cáo và hướng dẫn chi tiết từ A-Z một công cụ để bạn có thể tự tin thuyết trình trước nhóm và lớp.

1. CI/CD & Test-Harness Engineering là gì?

Để hiểu đơn giản, hãy hình dung việc phát triển phần mềm giống như một dây chuyền sản xuất lắp ráp ô tô:

CI (Continuous Integration - Tích hợp liên tục): Là quá trình tự động hóa việc ghép nối các đoạn code của nhiều thành viên trong nhóm lại với nhau. Mỗi khi có ai đó đẩy (push) code mới lên kho lưu trữ chung, hệ thống sẽ tự động "lắp ráp" (build) và chạy các bài kiểm tra (test) ngay lập tức để đảm bảo rằng mọi thứ vẫn hoạt động như mong đợi. Nếu có lỗi, hệ thống sẽ báo ngay để nhóm có thể xử lý kịp thời.

Tiếp tục cuộc trò chuyện này

CD (Continuous Delivery/Deployment - Phân phối/Triển khai liên tục): Sau khi CI chạy thành công (code đã an toàn), CD là bước tự động đóng gói và đưa phần mềm đó lên các môi trường thử nghiệm (staging) hoặc môi trường thực tế (production) để người dùng sử dụng.

Test-Harness Engineering (Kỹ thuật xây dựng môi trường kiểm thử): Nếu các bài test là những "bài thi", thì Test-Harness chính là "phòng thi, giám thị và giấy thi". Nó là tập hợp phần mềm, môi trường, dữ liệu giả lập và các công cụ giúp tự động hóa việc thực thi các kịch bản test, ghi nhận kết quả (Pass/Fail) và báo cáo. Nó cung cấp "đầu vào" (input data) cho phần mềm và đo lường "đầu ra" (output/behavior) xem có đúng kỳ vọng không.

Tóm lại: Chủ đề này nói về việc xây dựng một "đường ống" (pipeline) tự động: Lấy code -> Chạy Test tự động trong một môi trường chuẩn (Test-Harness) -> Đưa phần mềm lên server an toàn.

2. Đề xuất nội dung báo cáo cá nhân

Dựa trên yêu cầu của bạn, đây là dàn ý chi tiết cho 3 đầu mục:

Mục 1: Tìm hiểu chủ đề seminar (Tổng quan & Giá trị cốt lõi)

Định nghĩa ngắn gọn CI/CD và Test-Harness.

Tại sao dự án phần mềm hiện đại bắt buộc phải có CI/CD? (Giảm rủi ro, phát hiện bug sớm, không phải deploy thủ công dễ sai sót).

Mối quan hệ giữa CI/CD và môn học "Kiểm thử phần mềm" (Tự động hóa việc chạy các Test Case đã viết).

Mục 2: Tìm hiểu các tools phục vụ chủ đề

Bạn nên chia công cụ thành 2 nhóm để báo cáo rõ ràng hơn:

Nhóm Tools cho CI/CD (Pipeline Builders):

GitHub Actions: Phổ biến nhất hiện nay, tích hợp thẳng vào GitHub, rất thân thiện cho người mới.

GitLab CI/CD: Cực kỳ mạnh mẽ, được các công ty lớn dùng nhiều.

Nhóm Tools cho Test-Harness (Testing Frameworks & Environments):

Jest / Mocha: Rất tốt cho hệ sinh thái JavaScript/Node.js/React.

Selenium / Cypress: Dùng để giả lập thao tác người dùng trên trình duyệt (E2E Testing).

Postman / Newman: Chuyên dùng để test API tự động.

Mục 3: Cách thực hiện tool để hoàn thành chủ đề

(Trình bày chi tiết phần hướng dẫn thực hành ở mục số 3 ngay bên dưới).

3. Hướng dẫn thực hành Tool từ A-Z (Dành cho người mới)

Công cụ tôi đề xuất cho bạn là **GitHub Actions** kết hợp với **Jest** (một testing framework).

Sự kết hợp này rất thân thiện, hiệu quả, trực quan và đặc biệt phù hợp nếu bạn đang làm quen với các ứng dụng Web Fullstack (như Node.js). Bạn sẽ chứng minh được cho nhóm thấy: **Làm sao để mỗi khi dev đẩy code lên GitHub, hệ thống tự động chạy Test và báo xanh/đỏ.**

Bước 1: Chuẩn bị Input (Code & Test Case)

Tạo một thư mục dự án Node.js đơn giản trên máy tính của bạn và cài đặt Jest:

Bash



```
npm init -y
npm install --save-dev jest
```

Tạo file `math.js` (Chức năng cần test):

Tiếp tục cuộc trò chuyện này

JavaScript



```
function sum(a, b) {  
  return a + b;  
}  
module.exports = sum;
```

Tạo file `math.test.js` (Bài test - Test Harness):

JavaScript



```
const sum = require('./math');  
  
test('Cộng 1 + 2 phải bằng 3', () => {  
  expect(sum(1, 2)).toBe(3);  
});
```

Trong file `package.json`, sửa lại phần scripts:

JSON



```
"scripts": {  
  "test": "jest"  
}
```

Bước 2: Tạo luồng CI/CD với GitHub Actions

Ngay trong thư mục dự án đó, bạn tạo một cấu trúc thư mục đặc biệt như sau:

`.github/workflows/ci.yml`. GitHub sẽ tự động đọc thư mục này.

Copy đoạn cấu hình sau vào file `ci.yml`:

Tiếp tục cuộc trò chuyện này



```

name: Node.js CI Pipeline

# Trigger (Kích hoạt): Chạy tự động mỗi khi có code đẩy lên nhánh 'main'
on:
  push:
    branches: [ "main" ]

jobs:
  build-and-test:
    runs-on: ubuntu-latest # Môi trường chạy (như một server ảo)

    steps:
      - name: Checkout code
        uses: actions/checkout@v3 # Lấy code từ repo về server ảo

      - name: Setup Node.js
        uses: actions/setup-node@v3
        with:
          node-version: '18.x' # Cài đặt Node.js môi trường

      - name: Install Dependencies
        run: npm install # Cài các thư viện cần thiết (Jest)

      - name: Run Automated Tests
        run: npm test # Chạy lệnh test

```

Bước 3: Đẩy lên GitHub và quan sát Output

1. Bạn commit toàn bộ code và push lên một Repository trên GitHub.
2. **Output:** Mở kho lưu trữ GitHub của bạn, chuyển sang tab **"Actions"**.
3. Bạn sẽ thấy một tiến trình (workflow) đang chạy. Nhấn vào đó, bạn sẽ thấy GitHub Actions đang thực hiện từng bước (Checkout code -> Cài Node -> Chạy Test).
4. Nếu code đúng, toàn bộ quy trình sẽ hiện **Tích Xanh (Pass)**.

Bước 4: Demo thực tế cho nhóm (Cách tạo sự "Wow")

Trường hợp lỗi (Bắt Bug): Bạn sửa file `math.js` thành `return a - b`; và push code lên. Tab Actions trên GitHub sẽ chạy và báo **Dấu X Đỏ (Fail)**. Bấm vào log, nó sẽ chỉ đích danh: *"Kỳ vọng là 3 nhưng kết quả trả về là -1"*.

Ý nghĩa: Bạn giải thích với nhóm rằng: *"Nhờ CI/CD và Test-Harness, chúng ta không cho phép code lỗi được gộp vào dự án chung, chặn đứng bug ngay từ trên server trước khi đưa lên production."*

Bạn thấy hướng dẫn với C
thuyết trình cho nhóm ch
bài báo cáo phong phú h

Ở tab action trên Github repo của tôi báo là: "build-and-test
Node.js 20 is deprecated. The following actions target
Node.js 20 but are being forced to run on Node.js 24:
actions/checkout@v3, actions/setup-node@v3.", d

Chào bạn, đừng quá lo lắng! Đây là một tình huống cực kỳ phổ biến trong môi trường CI/CD thực tế và là một cơ hội tuyệt vời để bạn hiểu sâu hơn về cách các hệ thống DevOps vận hành.

Việc gặp lỗi về môi trường ngay cả khi code đúng là chuyện "cơm bữa", và xử lý được nó sẽ giúp bài báo cáo của bạn ghi điểm tuyệt đối.

Dưới đây là lời giải thích về bản chất và cách khắc phục:

1. Vì sao bạn gặp lỗi này?

Để hệ thống GitHub Actions hoạt động, bản thân nó cần chạy các "tiện ích" (như `actions/checkout` để lấy code, hay `actions/setup-node` để cài đặt môi trường). Các tiện ích này được viết bằng mã nguồn và cần chạy trên một nền tảng Node.js của **riêng máy chủ GitHub**, hoàn toàn tách biệt với Node.js trong dự án của bạn.

Nguyên nhân cốt lõi: Các phiên bản tiện ích cũ (`@v3` mà tôi hướng dẫn bạn trước đó) được thiết kế để chạy trên các phiên bản Node.js cũ. Gần đây, GitHub đã liên tục nâng cấp máy chủ (`ubuntu-latest`), họ thông báo ngừng hỗ trợ (deprecated) Node.js 20 cho các Action và ép các tiện ích `@v3` này chạy trên phiên bản Node.js 24 mới nhất.

Tiếp tục cuộc trò chuyện này

Kết quả: Việc "áp dụng" này gây ra sự không tương thích (vung đất) trong hệ thống

của bạn không sai.

Điều này cũng là một câu chuyện thực tế rất hay để bạn chia sẻ với Thoại, Phúc, Huy và Hải trong lúc làm báo cáo hoặc demo: *Trong CI/CD, đôi khi lỗi không đến từ code của lập trình viên, mà đến từ việc môi trường máy chủ (Infrastructure) thay đổi, đòi hỏi chúng ta phải liên tục cập nhật công cụ.*

2. Cách khắc phục

Cách giải quyết rất đơn giản: Bạn chỉ cần "nâng cấp" phiên bản của các tiện ích này lên bản mới nhất (@v4) để chúng tương thích hoàn toàn với máy chủ hiện tại của GitHub.

Bạn hãy mở file `.github/workflows/ci.yml` và sửa lại phần `steps` như sau (chú ý thay số 3 thành số 4):

YAML



```
name: Node.js CI Pipeline

on:
  push:
    branches: [ "main" ]

jobs:
  build-and-test:
    runs-on: ubuntu-latest

    steps:
      - name: Checkout code
        uses: actions/checkout@v4 # ĐÃ NÂNG CẤP LÊN v4

      - name: Setup Node.js
        uses: actions/setup-node@v4 # ĐÃ NÂNG CẤP LÊN v4
        with:
          node-version: '20.x' # Bạn có thể dùng 18.x hoặc 20.x cho dự án của bạn

      - name: Install Dependencies
        run: npm install
```

Tiếp tục cuộc trò chuyện này

- **name:** Run Automated Tests
run: npm test

Sau khi sửa xong, bạn lưu lại (commit) và đẩy (push) code mới này lên GitHub. Pipeline sẽ chạy lại và báo tích xanh.

3. Một lưu ý nhỏ (Kinh nghiệm thực tế)

Đôi khi, thông báo *"Node.js 20 is deprecated..."* chỉ là một **cảnh báo (Warning - thường có màu vàng)** chứ không phải là lỗi gây hỏng pipeline (Dấu X đỏ).

Nếu sau khi bạn cập nhật lên @v4 mà tab Actions vẫn báo Dấu X đỏ, bạn hãy bấm trực tiếp vào bước `Run Automated Tests` để xem log. Rất có thể do trong lúc test thử, file `math.js` đang vô tình bị viết sai (ví dụ code đang là `return a - b` nhưng test lại kỳ vọng là phép cộng) dẫn đến lệnh `npm test` bị rớt.

Bạn hãy thử cập nhật file `ci.yml`, **nhắc chắc chắn pipeline của bạn sẽ chạy mượt mà trở lại!**

[Chính sách quyền riêng tư của Google](#)
[Điều khoản dịch vụ của Google](#)
[Quyền riêng tư của bạn và Các ứng dụng Gemini](#)

Tiếp tục cuộc trò chuyện này